K215: verified proof refinements

Certificate composition, decoder context propagation,
and selection that permits early halting

Research and verification packet for Vladimir Reshetnikov's ProveIt

21 October 2026

Abstract

The most effective immediate simplification is to retain the accepted 215-state machine and reduce repeated proof obligations at its interfaces. This packet implements three such steps. First, a kernel-checked Rocq/Coq adapter combines the incoming 1,426-entry normalized source certificate with balanced lookup, preserving the public source-equivalence theorem. Second, a Lean theorem derives every internal decoder path from one initial marker invariant, removing separate route and intermediate tape-window premises. Third, an early-halting selection theorem replaces the nonfalsity requirements used only to ensure successful extraction of a row from the AX stack.

These are new checked components, not a freshly rebuilt complete K215-to-ZFC endpoint. The report distinguishes their exact boundaries from the repository's existing full endpoint. It compares alternative 215-state machines and records five additional bounded searches, including the final depth-41 R41 table omitted by its original one-row fold census. No verified 214-state construction was found; the search failures establish no lower bound.

1 Result and current baseline

The repository was inspected at

`aa66d0a33f1f8237a348899eb8095b045a502832`.

This matters because several attractive simplifications were already present either in maintained sources or in the three newly submitted archives. The accepted K215 already has Lean and Rocq correctness endpoints. Its phase-five Lean record reports a 184-module closure, with 37 affected modules freshly compiled and 147 previously checked objects reused after exact dependency and source binding. The phase-five work already removes zero offsets from the Lean source certificate and shares several semantic arguments [1, 2]. Those changes are background, not claimed as contributions of this packet.

The three incoming packets at the inspected commit contain further results: normalized source correspondence, balanced lookup for the older certificate, arbitrary-tape local decoder blocks, concrete zero-credit enumeration, and small table-to-table simulations. Their explicit remaining boundaries guided the work here [3, 4, 5].

Component	New result in this packet	Remaining boundary
Source correspondence	Balanced lookup composed with the normalized 1,426-entry checker; exact all-register theorem and public facade compiled	Production Rocq 9.1 endpoint rebuild
Decoder	All internal source paths follow from one marker and actual pointer bits; no per-edge window proofs	Maintained-definition identity, primitive edges, returns
AX selection	A generic row-selection theorem permits early halting and preserves the counter and opaque stack tail	Integration scope detailed in Section 5
State reduction	Five bounded searches on exact current tables, with complete receipts	No 214-state construction; no lower bound

The recommendation is to keep primary K215 as the main formal endpoint. For a second 215-state theorem, a small marker-transfer alternative is the least demanding incremental proof. A zero-credit machine has a cleaner independent enumeration argument, but changing the scheduler creates a larger integration task than retaining the already verified program.

1.1 What the checked statements mean

The desired mathematical statement remains

$$\mathsf{ZFC} \vdash (\neg\,\mathrm{Halt}(K215, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathsf{ZFC})).$$

It identifies nonhalting with syntactic consistency; it does not assert either side unconditionally. The present proof refinements preserve the source programs or the relevant execution contracts. They introduce no new set-theoretic assumption.

A theorem about a source program, a finite decoder prefix, or a command macro is not by itself a complete binary-machine endpoint. The compiled results below state which interface they cover. Likewise, a checked external Lean or Rocq theorem is not a new internal PA derivation. The repository's separate PA status is unchanged.

2 Simplify the interfaces before changing the calculus

The current endpoint factors through four different kinds of reasoning:

$$\begin{aligned}
\mathrm{Halt}(K215, \mathrm{blank}) &\longleftrightarrow \mathrm{Halt}(p_{215}, R_0),\\
\mathrm{Halt}(p_{215}, R_0) &\longleftrightarrow \mathrm{Halt}(\mathrm{dagPerm}, R_0),\\
\mathrm{Halt}(\mathrm{dagPerm}, R_0) &\longleftrightarrow \text{a finite refutation in the calibrated calculus},\\
\text{a finite calculus refutation} &\longleftrightarrow \text{a finite ZFC refutation}.
\end{aligned}$$

The first line contains the physical tape, initialization, and compiler simulation. The second is source correspondence. The third is the enumeration and proof-compilation argument. The last is logical calibration.

These layers should communicate through compact theorems. Repeating a source simulation, a tape-context reconstruction, or a list of syntactic nonfalsity facts in each client increases the number of obligations without strengthening the endpoint.

The guiding method in this packet is *checked refinement*. A new representation or finite checker is related to an existing sound interface by a small theorem. An untrusted generator may compute

a tree, role table, or certificate; the kernel checks the finite conditions on its actual output. Correctness does not depend on trusting the generator.

This approach also makes future machine changes cheaper to assess. A new table can reuse source and logical calibration if it satisfies the same execution contracts. Conversely, a new calculus must supply its own refutation translations; a shorter source listing alone does not establish equiconsistency.

3 Balanced lookup for the normalized source certificate

3.1 The two existing refinements

The incoming normalized checker identifies instruction pointers that differ only by trailing zero bits. With zero padding understood,

$$P \approx Q \quad \Longleftrightarrow \quad \forall i \in \mathbb{N},\ \mathrm{bit}(P, i) = \mathrm{bit}(Q, i).$$

This removes duplicate checkpoint representations, reducing the ordinary certificate from 2,240 entries to 1,426. The incoming proof transports completed segment endpoints through the established pointer-equivalence theorem. It does not blindly replace pointer arithmetic by arithmetic on trimmed lists.

Between two guarded decrement checkpoints, a finite segment accumulates increments. The checker compares the resulting register-increment counts and the next normalized pointers. At a decrement checkpoint it checks the same register and both branches. The soundness theorem quantifies over every natural-valued register file, not only reachable small values.

The other incoming refinement stores the older 2,240-entry certificate in a balanced binary tree. Its benefit is lookup work: an index is resolved in a number of tree nodes proportional to the tree height, rather than by walking a linked-list prefix. The earlier packet explicitly left composition with the normalized checker as further work [4].

3.2 A small refinement theorem composes them

A certificate tree has empty, leaf, and internal-node constructors. An internal node stores a binary natural number giving the size of its left subtree. Let $\mathrm{flat}(T)$ be the in-order list of entries. A validity predicate checks that every stored cut equals the actual left subtree size.

The inherited tree library proves

$$\mathrm{lookup}(T, j) = \mathrm{nth_error}\big(\mathrm{flat}(T), \mathrm{toNat}(j)\big) \qquad \text{when } T \text{ is valid.}$$

Out-of-range indices are included: both sides return failure. No default certificate entry is silently accepted.

The new `BalancedNormalized.v` uses tree lookup inside the normalized branch and entry checkers. It proves the following exact Boolean equality.

Theorem 3.1 (Balanced normalized refinement)**.** *For every valid certificate tree T and source trees t_1, t_2,*

$$\mathrm{nbcheck}(t_1, t_2, T) = \mathrm{ncheck}\big(t_1, t_2, \mathrm{flat}(T)\big).$$

Consequently, successful tree validity and successful balanced normalized checking imply the existing all-register source halting equivalence.

Proof. Replace tree lookup by list lookup in each branch. The two normalized pointer comparisons and the register-count comparison are unchanged. The entry equality follows by inspecting the two source instructions; the decrement case retains both branches. Tree-wide conjunction equals list-wide conjunction on the flattened entries. Apply the existing one-sided normalized checker theorem, whose mirror argument supplies the reverse implication. □

The adapter is short because it adds no new operational semantics. The unbounded simulation proof remains the already checked normalized one.

3.3 The exact K215 instance

The concrete tree is generated from the existing `Normalized215` literal. Separate kernel-checked statements establish:

$$\mathrm{flat}(T_{215}) = \mathrm{Normalized215},$$
$$\mathrm{validb}(T_{215}) = \mathrm{true}, \qquad \mathrm{size}(T_{215}) = 1426, \qquad \mathrm{height}(T_{215}) = 12.$$

Every lookup has at most 12 visited tree nodes. The theorem about height includes arbitrary indices; the actual checker additionally rejects failed lookups.

The resulting exact source theorem is

$$\forall R : \mathbb{N} \to \mathbb{N}, \quad \mathrm{rm_halts}(p_{215}, ([], R)) \longleftrightarrow \mathrm{rm_halts}(\mathrm{dagPerm}, ([], R)).$$

The facade exports the existing name `K215F.Source215.p215_dagPerm`. An explicit typed caller compiled against that facade. The maintained `Correct215.v` uses precisely this interface when composing source and logical correctness [6].

Both literal source files, `Prog215.v` and `DagPerm.v`, were fetched at the inspected commit and compared byte for byte with the compiled inputs. The package retains their hashes and the current public boundary in `balanced_normalized/current-binding.json`.

3.4 Exact lookup measurements

The following census counts only the data-structure nodes visited by certificate endpoint lookup. It does not count source-tree decoding, segment evaluation, proof elaboration, tree construction, or kernel work outside this operation.

Representation	Entries	Lookups	Visited nodes	Maximum tree nodes
Ordinary list	2,240	4,479	5,018,706	–
Ordinary balanced tree	2,240	4,479	54,514	13
Normalized list	1,426	2,851	2,033,059	–
Normalized balanced tree	1,426	2,851	32,961	12

There are 1,425 decrement entries and one halt entry in the normalized certificate. Thus the root check plus two references per decrement gives $1 + 2(1425) = 2851$ lookups.

The normalized data file is 108,633 bytes, compared with 171,678 bytes for the ordinary compact data file. The shared path codec adds 1,325 bytes. These are certificate-source measurements, not total proof-library sizes. Balanced storage reduces a cost inside checking; it does not by itself reduce the number of machine states or prove a whole-build speedup.

3.5 Fresh verification and scope

The isolated package freshly compiled 17 modules with Coq 8.18.0 and OCaml 4.14.1. The nine assumption reports printed by the new adapter, concrete tree statements, exact theorem, facade, and client are closed under the global context. The summed compiler time in this execution was 50.941 seconds; the exact `Source215Composed.v` compilation took 9.126 seconds. These are observations of this run, not a benchmark comparison.

A separate recursive `coqchk -silent EndpointClient` completed successfully in 306.215 seconds. It checked the concrete client and its recursive dependencies, rather than only the abstract tree library.

The compatibility route adds the supplied `Compat818` import to the inherited `RMEquiv.v` source. That exact compiled source is hashed in the receipt. No new axiom is introduced. A maintained Rocq 9.1 build and the full binary/ZFC endpoint were not rebuilt here. The package includes the production-toolchain branch of the reproduction recipe, but it is not represented as executed evidence.

4 Derive decoder paths from one invariant

4.1 What the local-block proof still required

The incoming decoder package already proves 281 local finite-window blocks for arbitrary complete tapes. It also proves a chain-composition theorem. However, a client of that chain theorem still had to supply the semantic window premise for each successive block. Enumerating the blocks does not itself prove that their required tape contexts arise on a real source path.

Two modules close this obligation. The generic proof is in `DecoderContext.lean`. Its concrete instance is in `K215Context.lean`. They use the three unchanged incoming foundations. A typed audit module makes six fresh compilations in the delivered check.

4.2 Finite contexts and their propagation

A decoder role consists of a source node, a physical state, a finite suffix immediately preceding the head, and its two outgoing edges. Different roles may use the same physical state in different contexts.

Write Before(s) for the finite partial tape that places a suffix s at positions $-|s|, \dots, -1$. For the current input bit b, the known local context is

$$K(r, b) = \mathrm{Before}(\mathrm{suffix}(r)) \cup \{0 \mapsto b\}.$$

A finite coverage check confirms that every cell required by a block's input window occurs in $K(r, b)$ with the same value. A second coverage check confirms that $K(r, b)$ supplies the next role's suffix after the head moves one cell to the right. Entry and exit states are checked too.

These conditions compose because the inherited local theorem restores the *entire* tape. Intermediate head movements and writes are allowed; their net effect is the certified state change and the one-cell head advance. Consequently the next suffix can be proved from the same original tape.

Lemma 4.1 (One internal edge)**.** *Suppose the local block and its finite context checks pass. If the complete tape matches the current suffix and current bit at head position a, the block runs for positive time, ends in the next role at $a + 1$, restores the whole tape, and establishes the next suffix invariant.*

Proof. Finite cell inclusion supplies the local block's arbitrary-tape window premise. Apply the inherited block theorem. Since its output tape is identical to its input tape, the second finite inclusion supplies the shifted suffix. Positivity comes from the checked block duration. □

The checker verifies these conditions for all 371 internal edges in the 234-role product, using 742 context inclusions. Primitive edges are handled separately and are deliberately outside this check's semantic claim.

4.3 A source-path theorem without a route witness

A further finite check links the role graph to the supplied 202-node source DAG. Whenever a source child is internal, the corresponding role edge must be internal and point to a role for that child. This proves that every internal source path has a decoder route. It removes even an explicit route or successful-follow premise from the final theorem.

Let $\mathrm{Reads}(T, a, w)$ mean that the successive tape cells starting at a contain the bits of w. The exact root role has physical state 84 and suffix [1].

Theorem 4.2 (Every internal K215 source path)**.** *Suppose w follows the supplied source DAG from its root to an internal node m. For every integer a and complete Boolean tape T, assume*

$$T(a-1) = 1, \qquad \mathrm{Reads}(T, a, w).$$

There exist a role f and a duration n such that f's source node is m, and execution of the supplied literal K215 table satisfies

$$(84, a, T) \xrightarrow{n} (f.\mathrm{state}, a + |w|, T).$$

The next-role suffix invariant holds, and $w \neq []$ implies $n > 0$.

Proof. Induct on the source path, using the finite source-link check to obtain the successive role edges. Apply the one-edge lemma at each step and compose finite executions. Tape restoration supplies all subsequent pointer bits and suffix facts. Nonempty paths contain a positive-duration block. The empty path has duration zero. □

The compiled theorem is `K215Context.every_internal_source_path`. Its tape quantifier is unrestricted outside the initial marker and supplied pointer bits. It does not assume a finite-support tape, a register bound, or a successful finite simulation of some particular run.

4.4 Concrete checking and remaining adapter

The six-module Lean 4.32.0 build completed with zero warnings and only the standard Lean axioms in the printed semantic theorem reports. No `sorryAx` or `native_decide` is used. Five checked malformed controls reject a missing predecessor fact, a wrong predecessor fact, a wrong next suffix, a missing target role, and a primitive source node used as an internal parent.

An independent finite audit binds every published transition row and every expanded source leaf to the supplied constants: 430 rows and 4,751 leaves. It also checks the 4,750 internal source occurrences, including the root. The longest internal prefix has 46 bits and takes 55 steps in this exact decoder. These finite measurements corroborate the literal instance; the universal path statement is the kernel theorem.

This is a substantial strengthening of the incoming chain interface, but it is not yet a replacement for the whole runtime proof. Three tasks remain:

1. Prove in the maintained kernel namespace that the supplied native table and source-DAG constants denote the maintained K215 definitions. The delivered external equality audit is exact, but is not that kernel adapter. Transport the complete integer-indexed tape semantics to the maintained tape representation as part of this binding.
2. Connect the final primitive-entry edges to the existing primitive register-operation contracts. Theorem 4.2 ends at an internal source node.
3. Reuse the return and register-frame theorems to reestablish the root marker, and retain the exact blank initialization proof.

The value of the new theorem is that none of these clients needs to rebuild intermediate decoder window proofs for every occurrence.

5 Selection may halt early

5.1 The unnecessary strength in the old selection argument

A completeness compiler needs a block that either reaches its requested formula configuration or halts earlier. Early detection of the falsity code already achieves its purpose. Proving that every intermediate axiom or gate code is different from falsity is therefore unnecessary when those facts are used only to force normal return from a selector.

This distinction is especially useful for the AX bundle: selecting one row should be a generic stack operation. It should not require a separate syntactic proof about every skipped row. The theorem below allows arbitrary row values, including non-formula codes and the falsity code itself.

5.2 The exact command semantics

Write a numeric machine state as (T, W, D, H), representing top, stack, counter, and cache. Let $P(x, W)$ be stack push, and let unP be its inverse, with the established law

$$\mathrm{unP}(P(x, W)) = (x, W).$$

The literal production command shapes are [8]

$$\mathsf{NIP}(T, W, D, H) = \begin{cases} \text{halt}, & H = 1, \\ (T, \pi_2(W), D, \pi_1(W)), & H \neq 1, \end{cases}$$

and

$$\mathsf{DROP}(T, W, D, H) = (H, P(T, W), D, H).$$

In particular, NIP tests the *old cache before popping*. This timing determines the correct selection counts.

Let $\mathrm{pack}(L, W)$ push the explicit list L above the opaque numeric tail W, with the first listed value popped first.

Theorem 5.1 (Early-halting indexed selection)**.** *Let $L = [x_0, \ldots, x_{r-1}]$ and $0 \leq j < r$. From state $\big(G, \mathrm{pack}(L +\!\!+ [0], W), D, H\big)$, the command word*

$$\mathsf{NIP}^{\,j+1} \;;\; \mathsf{DROP} \;;\; \mathsf{NIP}^{\,r+1-j}$$

either halts early or returns exactly $(x_j, W, D, 0)$. There are no theoremhood, formula-well-formedness, nonfalsity, or fresh-variable premises on G, H, or the entries of L.

Proof. If any alarm occurs, the first alternative holds. Otherwise, the first $j+1$ NIPs leave x_j in the cache and remove the preceding rows and x_j from the explicit stack prefix. DROP makes x_j the top, retains it in the cache, and pushes the old top G.

The remaining explicit prefix now consists of G, the $r-1-j$ remaining rows, and the final zero: exactly $r+1-j$ values. If no alarm occurs during these NIPs, their last pop places zero in the cache. The top remains x_j, the counter remains D, and the stack is exactly W. The tail W is never popped. The final zero is loaded but is not tested again within this block. □

The generic proof uses only the push/pop inverse law. It does not rely on a separately stored stack length, so numerical aliases such as $P(0,0) = 0$ cause no problem. The explicit-prefix calculation is justified by repeated applications of that inverse law.

The extra zero is essential to the zero-cache conclusion. Replacing it by an arbitrary nonzero sentinel changes the final cache on a normal run. The finite corroboration includes a counterexample to that tempting overgeneralization.

5.3 Exact binding to the maintained command language

This packet goes beyond a second, separately defined command semantics. The compiled `K215SelectorAdapter.v` proves:

- the explicit embedding of NIP and DROP into the production `L1_CompPerm.cmd`;
- exact one-step and execution equalities for that embedding;
- the concrete selection-word equality;
- the exact nine-row view of production `EnumPerm.ax_stack`;
- the actual-command theorems `select_actual` and `select_AX_row`.

The nine rows, in pop order, are

$$[\rho_1(a,b), \rho_2(a,b,c), \rho_3(a,b,c), \rho_4(a,b,c),\\ \rho_5(a,b,c), \rho_6(a,c), \mathrm{growG}(a,b,c), \rho_8(a,b,c,u), \mathrm{wal}(u,h)].$$

They are followed by the saved old top. After the existing reset/discard setup, that saved top is zero. The eight axiom selectors use indices $0,\ldots,7$; GEN uses index 8 after its existing setup establishes the same zero-bottom condition. For these nine rows every selector has $9+3=12$ commands, although the number of NIPs on each side of DROP varies.

The production theorem states that, from the actual AX stack with saved top zero, production `exec` either returns `None` or returns the selected row with the original tail and zero counter/cache. No definition of the production commands or AX stack is changed.

5.4 Verification and how to adopt the result

The two generic modules compiled and passed recursive `coqchk`. A second fresh build compiled all eleven modules in the production adapter's small dependency closure, including the current command compiler and enumeration definitions, and recursively rechecked `K215SelectorAdapter`. The two new actual-command assumption reports are closed.

Five current production source files were bound to their GitHub blobs at the inspected commit. For the Coq 8.18 compatibility run, three receive only an import-prefix adjustment from `Stdlib` to `Coq`. The inherited optimized dominated-rho8 proof is explicitly credited and freshly compiled in this dependency closure. The earlier failed adapter attempt, due to a missing explicit import, is retained separately from the successful final receipt.

The independent numeric check covers 11,502 finite cases: 10,326 early halts and 1,176 exact returns. It checks zero aliases, small arbitrary tails, counter preservation, and the sentinel counterexample. These cases are corroboration; the universal theorem and its actual-command binding are kernel-checked.

The next production edit is small in scope: retain the exact discard/setup contracts and replace the eight row-extraction arguments and GEN extraction tail with `select_AX_row`. The declared completeness contract already allows early halting. Keep nonfalsity lemmas used by exact-return clients or elsewhere in soundness.

The current leaf proofs already use a compact tactic, so a reduction in total source lines or kernel time has not been established. What is proved is that the stronger nonfalsity obligations are unnecessary on this selection path. The full maintained `build` theorem and final binary/ZFC endpoint have not been replaced or rebuilt.

6 Which other 215-state candidate is easier to prove?

There are three different meanings of *easier* here: smallest additional proof over the accepted endpoint, cleanest independent enumerator, and simplest physical encoding. They point to different candidates.

Candidate	Best reason to use it	Qualification
Primary K215	Already has the complete formal endpoint; all three refinements above apply without changing its table	Best choice for reducing present maintenance work
M124 / FM215	Small marker-based table simulation transports the existing correctness argument	Representation bridge and endpoint composition still need their own checks
CM215	Incoming work already records a Lean Core bridge	Least remaining Lean adapter work may outweigh a more involved local invariant
ZC215	Zero-credit enumeration removes the reserve/debt calculation	Its physical runtime and changed scheduler still need full integration
R41	Shallower source addresses, same calculus and primitive control flow	Shorter addresses are not a smaller state count or a general speedup

6.1 The least additional proof: a marker-transfer machine

For a second theorem derived from primary K215, M124 or FM215 is a good choice. The incoming finite-table theorem already isolates two predecessor marker facts and a one-or-three-step simulation on arbitrary tapes [4]. It avoids proving a new calculus or scheduler.

The concrete family uses old roles 124 and 127. Incoming transitions establish predecessor markers 1 and 0, respectively. A shared role and a selector distinguish the two old zero branches by that marker, returning through existing provider states. The providers differ between the literal M124 and FM215 tables. The local simulation preserves the complete tape and head at each corresponding boundary and uses positive time.

The invariant concerns the immediately preceding cell at two control states, so it is far smaller than a global register-format invariant. Since the roles are noninitial and all their incoming transitions write the required marker, the relation starts from arbitrary tapes at state zero.

The alternative remains at 215 states because one role is merged but the selector occupies the

released role's state. The operation reorganizes the decoder; it does not yield a net deletion. Reporting it as a 214-state construction would miscount the selector.

These are existing alternatives, not new machines discovered in this packet. The incoming native table-to-table theorems do not by themselves constitute the repository's dense/zipper ZFC endpoint. CM215 may be faster to integrate in Lean because an incoming packet already records its Core adapter. The exact next endpoint should be chosen with that engineering status in mind.

6.2 The cleanest independent scheduler: ZC215

ZC215 assigns DROP the lower tag and NIP the next tag. Lowering a marked NIP occurrence to the corresponding DROP occurrence gives an injection into DROP occurrences belonging to earlier raw codes in the exhaustive schedule. Thus earlier DROP credits cover the potential NIP debits. The incoming packets now prove the concrete Cantor-code and decoder facts behind this argument, rather than leaving them abstract [3, 4].

This is a useful simplification of an independently reconstructed enumerator. It does not automatically minimize the total proof needed today, because primary K215 already has a complete endpoint. The combined GC-ZC alternative also changes the binder/compiler arrangement and hence introduces more interfaces to revisit.

The new early-halting selector is compatible with this general proof strategy, but it does not alter command tags or establish a new binary machine. A proof simplification and a state reduction remain different claims.

6.3 The encoding comparison: R41

The R41 source retains 202 distinct subtrees and 4,751 primitive occurrences. Its maximum address depth is 41 instead of 47, and the sum of address lengths is 149,265 instead of 160,305. Nineteen transition rows change, and the published source/runtime evidence remains at 215 working states [7].

The reduction of 11,040 address bits is a static improvement. The original R41 packet explicitly reports mixed runtime changes, so it should not be advertised as uniformly faster. Nor does a shallower tree make its correctness proof as small as a local transfer from the already checked primary table. It is nevertheless a useful source for testing a different allocation geometry, which motivated the new searches below.

7 The search for 214 states

No verified 214-state machine was constructed in this work. A useful coverage gap was found in the original R41 one-row fold census: it tested 32 earlier depth-44/45 tables and explicitly excluded the final depth-41 machine [7]. This packet tests the actual final table in that family, along with two newer layouts. Other incoming investigations have tested R41 under different families; no claim of first-ever R41 search is made.

Exact table and family	Bounds	Specifications
R41, one-row contextual	History depth 4; 45 steps; 200 symbolic nodes	71,380
R41, one-row contextual	History depth 8; 97 steps; 1,000 symbolic nodes	99,760
Shallow context-return215, one-row contextual	Depth 8; 97 steps; 1,000 symbolic nodes	108,360
GC-ZC215 canonical, one-row contextual	Depth 8; 97 steps; 1,000 symbolic nodes	97,180
R41, blank-window folds	Radius 7; 97 steps; specified inherited/common-action family	632,584

All five searches returned no survivor. The counts describe search specifications, not distinct complete machines; overlapping families should not be added and advertised as a count of unique constructions.

The first four families require the opposite-symbol transitions of each candidate state pair to be literally equal. They enumerate replacement writes, moves, and working-state targets and use bounded graph histories as local contexts.

The final search exhausts all 46,010 ordered pairs of distinct working states, including initial-state mergers. It considers 182,992 inherited action choices and 449,592 one-common-action choices. All 632,584 fail its finite-window common-endpoint filter; none reaches the independent acceptance checker. The observed execution finished before its inherited 110-second ceiling, so that ceiling did not truncate this recorded run. The complete compressed rejection record is included.

The three input tables and both main search entry points were fetched at the inspected GitHub commit and matched byte for byte to the executed copies. The source algorithms are inherited unchanged; the new contribution is the explicitly identified table coverage and preserved executions.

7.1 What these failures do not establish

Failure of a bounded local filter is not a proof of semantic inequivalence. A candidate may require a longer reconciliation path or a sharper invariant than the filter supplies. A history overapproximation may include impossible contexts and reject a sound machine. Larger coordinated edits, another source layout, and a different calibrated calculus remain outside these searches.

The results also do not replace the incoming packet's much larger coordinated two-merge/one-selector investigation. The families have different restrictions. None supplies a lower bound on the state count.

7.2 A concrete next search direction

The new decoder theorem suggests a better proof interface for future allocation experiments. Search can operate on roles together with finite suffix contexts. A successful candidate should export:

1. a complete literal transition table, including blank initialization;
2. the source-node/physical-state/context role product;
3. positive-duration blocks with whole-tape effects;
4. finite context-inclusion and source-link checks;
5. primitive and return contracts plus the exact initial frame.

The kernel can then validate the candidate's actual contexts instead of requiring a fresh expanded proof for every source occurrence.

To get a net state saving from the two-marker family, the discriminator must be supplied without consuming the freed state, or a coordinated change must release another state elsewhere. The obvious short local versions are heavily searched. A useful new attempt should jointly vary source layout, physical allocation, and the context invariant, and should count the selector, providers, and initialization before claiming 214.

On the formalism side, the permission to generate junk codes is valuable only when the altered proof system still has explicit refutation translations in both directions. The selection theorem shows one place where junk values need no extra syntactic case analysis: if they trigger the designated halt, completeness has already succeeded. It does not license unsound inference rules or supply a new equiconsistency proof. No smaller calibrated calculus is claimed here.

8 Integration plan and exact acceptance gates

The new results are useful as a sequence of small changes.

1. Adopt the source facade first. Move the combined checker and data into the maintained Rocq source namespace, update the generator and dependency inventory, and preserve `K215F.Source215.p215_dagPerm`. Rebuild the production `Correct215` and `CorrectDense215` modules under the maintained toolchain and inspect their assumptions. The checked facade and client make this a concrete integration task; they do not replace that final production build.

2. Use the actual-command selector. Replace the eight leaf-extraction arguments and the GEN extraction tail with the compiled `select_AX_row` theorem. Retain the existing setup and exact discard contracts. Recheck the compiler's existing completeness statement and soundness dependencies. Measure total source/proof-object size or compile time only after that adoption if a performance claim is wanted.

3. Install the decoder adapter. Identify the literal DAG/table with maintained constants in the kernel and transport the full-tape semantics into the production tape model. Compose the all-internal-path theorem with primitive, return, and register contracts. Keep initialization separate. This is where substantial occurrence-by-occurrence context proof work can be removed, but the delivered theorem does not yet quantify that total reduction.

4. Prove one alternative by transport. For a second 215-state endpoint, use M124/FM215's local marker theorem or CM215's already advanced Lean bridge. Avoid rebuilding logical calibration when a table simulation preserves the existing source program's behavior. A clean ZC215 endpoint remains worthwhile as an independent scheduler implementation.

5. Accept a state record only at the full interface. A 214-state table must include initialization, have the exact working-state count, and satisfy an unbounded correctness argument for its own literal rows and initial tape. Finite startups, static DAG counts, or a bounded search survivor are not substitutes.

9 Contents and reproduction

The archive is organized as follows.

Path	Purpose
`report.tex`, `report.pdf`	Editable report and its compiled, visually checked PDF
`balanced_normalized/`	New composition, inherited dependencies, exact facade/-client, metrics, current-source binding, compiler and recursive-audit evidence
`decoder_context/`	New generic and literal Lean context proofs, inherited foundations, literal inputs, typed audit, and complete fresh-build evidence
`early_halt/`	Generic selector, numeric layer, exact production adapter, source provenance, fresh builds, and finite corroboration
`search214/`	Five bounded search runs, exact input tables, inherited algorithms, portable replay, and complete compressed rejection records
`reviews/`	Scoped independent reviews within this research collaboration
`MANIFEST.json`, `verify_packet.py`	Delivered-file integrity and recorded-evidence checks

The lightweight audit checks package integrity, recomputes lookup metrics, checks the recorded proof/source bindings, and reruns the small context and numeric corroborations. It does not silently invoke a full theorem-prover build or repeat the large searches.

```
python3 verify_packet.py
```

Fresh formal checks can be run separately, using empty output directories:

```
python3 balanced_normalized/build.py \
  --out /tmp/k215-balanced-normalized-fresh --recursive

python3 decoder_context/build.py \
  --lean /absolute/path/to/lean --out /tmp/k215-context-fresh

python3 early_halt/build.py --output /tmp/k215-selector-fresh
python3 early_halt/build_production.py \
  --output /tmp/k215-selector-production-fresh
```

The source package accepts Coq 8.18 and includes a Rocq 9.1 recipe branch; the executed evidence is from Coq 8.18. The decoder requires Lean 4.32.0. Toolchain installations and compiled proof objects are not included. The proof sources and the inherited dependency sources needed for these focused checks are included, along with their notices.

The optional search replay is:

```
python3 -B search214/reproduce.py --output /tmp/k215-fold-replay
```

Its blank-window component retains the inherited time ceiling. On a slower machine the driver may report that the family did not complete, rather than promoting an incomplete execution to the recorded completion claim.

Build the PDF with the included `build_report.py`, which invokes pdfLaTeX twice in a separate build directory. Historical logs retain their actual execution paths as provenance; reproduction entry points use package-relative inputs.

Attribution and result boundary

The accepted calculus, K215 machine, register compiler, earlier proof certificates, decoder blocks, alternative tables, and search algorithms retain their original authorship and licenses. The new contributions are the balanced/normalized composition, automatic internal decoder-context propagation, the early-halting selection theorem and exact production adapter, and the new explicitly scoped search executions. The three incoming packets were inspected before these extensions were developed.

Separate review here means another worker in this research collaboration, not an independent external human endorsement. Every assertion of formal checking is tied to the included source hashes, commands, return codes, and logs.

The strongest new result is a smaller set of reusable proof obligations for the existing machine. The main K215 endpoint remains inherited. There is no claimed 214-state machine, unconditional nonhalting theorem, or minimum-state theorem.

References

[1] ProveIt, *Smaller machines for Con(ZFC)*, inspected commit aa66d0a33f1f8237a348899eb8095b045a502832. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/README.md

[2] ProveIt, *Ordinary certificates and shared semantic lemmas*, phase-five checkpoint. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase5/README.md

[3] *Checked proof simplifications for K215*, incoming research packet, 21 October 2026. Source normalization, local decoder certificates, concrete zero-credit argument, and bounded coordinated state search. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/docs/incoming/K215_Checked_Simplifications_2026-10-21.zip

[4] *Verified certificate refinements for K215*, incoming research packet, 21 October 2026. Balanced lookup, shared marker certificates, concrete credit theorem, and restricted fold searches. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/docs/incoming/K215_Certificate_Refinements_2026-10-21.zip

[5] *Affine proof cores and incremental machine transport*, incoming research packet, 21 October 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/docs/incoming/Affine_Proof_Cores_and_Transport_2026-10-21.zip

[6] ProveIt, `K215F.Correct215`, maintained source-to-logical endpoint. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/Correct215.v

[7] ProveIt, *K215-R41: a rebalanced 215-state ZFC-consistency machine.* https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/candidate-k215/alternatives/rebalanced/README.md

[8] ProveIt, `L1_CompPerm.v`, maintained command compiler. https://github.com/VladimirReshetnikov/ProveIt/blob/aa66d0a33f1f8237a348899eb8095b045a502832/Computability/BusyBeaver/Research/ZFC/k225-formal/rocqperm/L1_CompPerm.v